

JAVA PROGRAMMING LANGUAGE

HANDWRITTEN NOTES — MASTERCLASS EDITION

Complete Placement & Core Java Developer Guide (111 Pages Converted Text)

TABLE OF CONTENTS / INDEX (ALL 13 MODULES)

- Module 1: History of Java & Core Features (Pages 1 – 5)
- Module 2: JVM, JRE, JDK & Program Generations (Pages 6 – 9)
- Module 3: Input Handling: Scanner, BigInteger & BigDecimal (Pages 10 – 14)
- Module 4: Java Expressions, Data Types & Flow Control (Pages 15 – 18)
- Module 5: Control Flow Statements (If-Else, Switch, Loops) (Pages 19 – 23)
- Module 6: Classes, Objects & Constructors (Pages 24 – 32)
- Module 7: Object-Oriented Programming (OOPs) Concepts (Pages 33 – 42)
- Module 8: Inheritance, Super Keyword & Method Overriding (Pages 43 – 52)
- Module 9: Polymorphism, Abstraction & Abstract Classes (Pages 53 – 62)
- Module 10: Interfaces, Default/Static Methods & Packages (Pages 63 – 72)
- Module 11: Exception Handling (Try-Catch, Custom Exceptions) (Pages 73 – 85)
- Module 12: String Handling, StringBuilder & StringBuffer (Pages 86 – 100)
- Module 13: Enums, Static Keyword & JVM Memory Model (Pages 101 – 117)

MODULES 1–3: JAVA CORE & INPUT HANDLING

JAVA PROGRAMMING LANGUAGE — HANDWRITTEN NOTES (111-Page Converted Edition)

1. History of Java & Core Attributes

- Initiated in June 1991 by James Gosling, Mike Sheridan & Patrick Naughton (Green Team).
- Originally named "Greentalk" (.gt file extension), later renamed Oak, and renamed Java in 1995.
- JDK 1.0 released on January 23, 1996.
- 12 Primary Attributes: Simple, Portable, Secure, High-Performance, Multithreaded, Interpreted, Platform-Independent, Dynamic, Architecture-Neutral, Object-Oriented, Robust, Backward-Compatible.

2. JVM, JRE, JDK & Bytecode Execution Flow

- JDK (Java Development Kit) = JRE + Development Tools (javac, javap).
- JRE (Java Runtime Environment) = JVM + Core Class Libraries.
- Execution Flow: .java source code -> Compiled by javac -> .class Bytecode -> Executed by JVM Interpreter/JIT.

3. Input Handling with Scanner & BigInteger/BigDecimal

```
import java.util.Scanner;
import java.math.BigInteger;

public class InputHandlingDemo {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a big integer: ");
        BigInteger val1 = input.nextBigInteger();
        BigInteger val2 = BigInteger.valueOf(500);
        BigInteger result = val1.add(val2); // BigInteger methods: add, subtract, multiply, divide
        System.out.println("Result: " + result);
    }
}
```

MODULES 6–8: CLASSES, OOPS & INHERITANCE

JAVA PROGRAMMING LANGUAGE — HANDWRITTEN NOTES (111-Page Converted Edition)

4. Classes, Objects & Constructors

- Class: Blueprint or template containing data members (fields) and methods.
- Object: Instance of a class created using the `new` keyword.
- Constructors: Invoked implicitly when object is instantiated. Same name as class, no return type.
- Types: No-Arg Constructor, Parameterized Constructor, Default Constructor (auto-generated if none defined).

5. Inheritance & `super` Keyword Implementation

```
class Box {
    private double width, height, depth;
    Box(double w, double h, double d) {
        this.width = w; this.height = h; this.depth = d;
    }
    double volume() { return width * height * depth; }
}

class BoxWeight extends Box {
    double weight;
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d); // Call superclass constructor (Must be 1st statement)
        this.weight = m;
    }
}
```

MODULES 9–11: ABSTRACTION, INTERFACES & EXCEPTIONS

JAVA PROGRAMMING LANGUAGE — HANDWRITTEN NOTES (111-Page Converted Edition)

6. Abstract Classes vs Interfaces (Java 8+)

- Abstract Class: Declared with `abstract`, cannot be instantiated directly, can have concrete & abstract methods.
- Interface: Fully abstract contract (`interface`), methods implicitly `public abstract`.
- Java 8 Features: Default methods (`default void getSides()`) & Interface static methods.

7. Exception Handling & Custom Exceptions

```
// Custom Exception Example
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) { super(message); }
}

public class ExceptionDemo {
    public static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) throw new InvalidAgeException("Age is less than 18!");
        System.out.println("Valid Age!");
    }
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (InvalidAgeException e) {
            System.err.println("Caught: " + e.getMessage());
        }
    }
}
```

MODULES 12–13: STRINGS, ENUMS & JVM MEMORY MODEL

JAVA PROGRAMMING LANGUAGE — HANDWRITTEN NOTES (111-Page Converted Edition)

8. String, StringBuffer & StringBuilder Comparison

- String: Immutable character sequence stored in String Constant Pool (Heap).
- StringBuffer: Mutable character sequence, Thread-safe (Synchronized methods).
- StringBuilder: Mutable character sequence, Not thread-safe, Faster performance for single-threaded tasks.

9. JVM Memory Model Areas (5 Main Areas)

1. Metaspace: Stores class metadata, bytecodes, static variables (unloaded when class is unloaded).
2. Heap: Stores all instantiated objects and instance variables (Managed by Garbage Collector).
3. Stack: Stores thread execution frames, local variables, method call states & references.
4. PC Register: Holds the memory address of the currently executing JVM instruction per thread.
5. Native Method Stack: Manages C/C++ native method execution state via JNI (Java Native Interface).

COMPLETION VERIFICATION & ANTI-INSPECT SECURITY

This document is the official converted text edition of Java Programming Language Handwritten Notes.
Cryptographically verified & delivered via serverless Razorpay payment verification.